SPORK

Self-Speculative Forking to Accelerate Agentic LLM Inference



Huajun Bai et al. • arXiv:2607.03333

汇报人：张帅 | Agent 动作投机 Survey | 2026.08

外部 Speculator 的成本，也是 Agent 投机的系统瓶颈

背景 / 核心问题

- 外部小模型需要第二份权重 /KV cache，且常与 Actor 的真实调用分布不一致。
- 历史 pattern 对长尾工具和少见参数容易失效。
- 研究问题：运行中的 LLM 能否足够早、足够准且失败后可回收地预测自己?

主要贡献

1. **D1 | Prefix-KV Fork**

首 token 后强制工具前缀，缓存命中将 probe 约 1.6s 降到 0.35s 。

2. **D2 | Confidence Gate**

用 probe 自身 tool-name logprob，选择最早可信提交时刻。

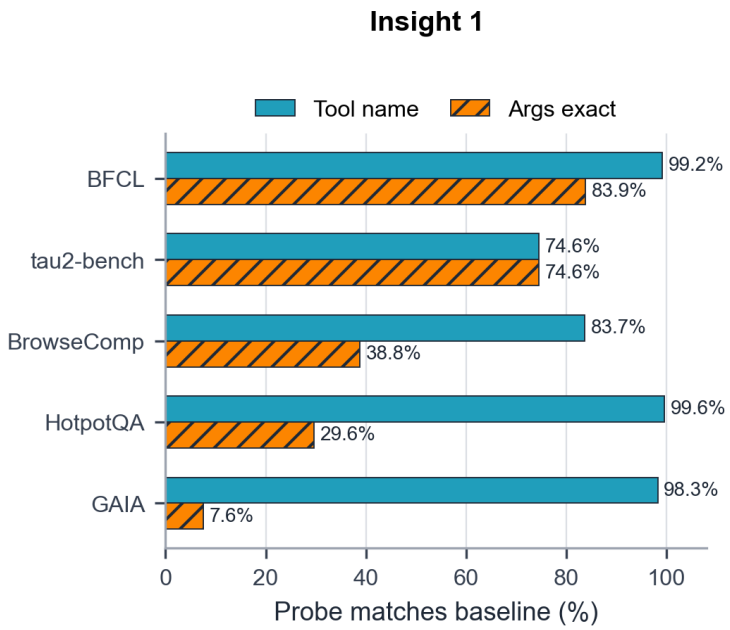
3. **D3 | Token Reuse**

严格拒绝时，把共享工具调用前缀当作 token 草稿。

Training-free self-speculation：同一 Actor 、共享 Prefix KV 、工具级严格匹配。

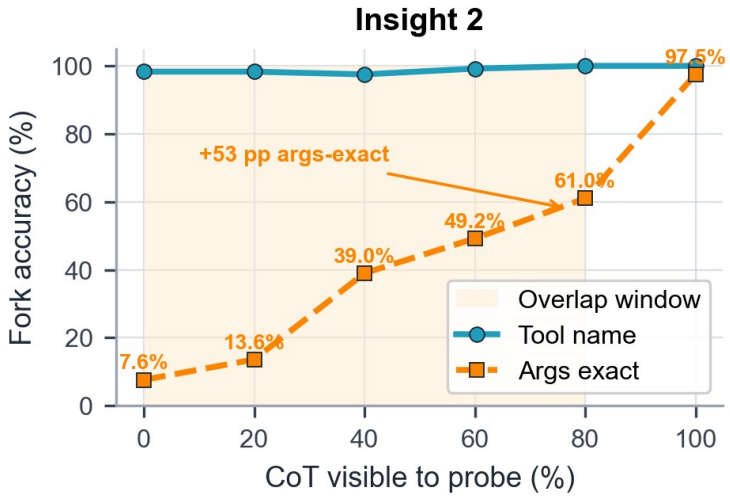
同一模型确实“早知道、知道何时可信、失败也不全浪费”

Insight 1 ｜早知道工具名



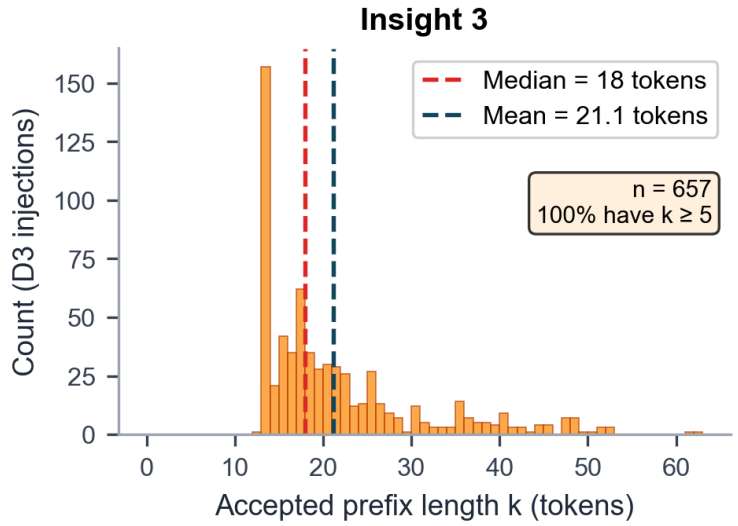
tool-name 74.6%–99.6%
args exact 7.6%–83.9%

Insight 2 ｜信心随 CoT 上升



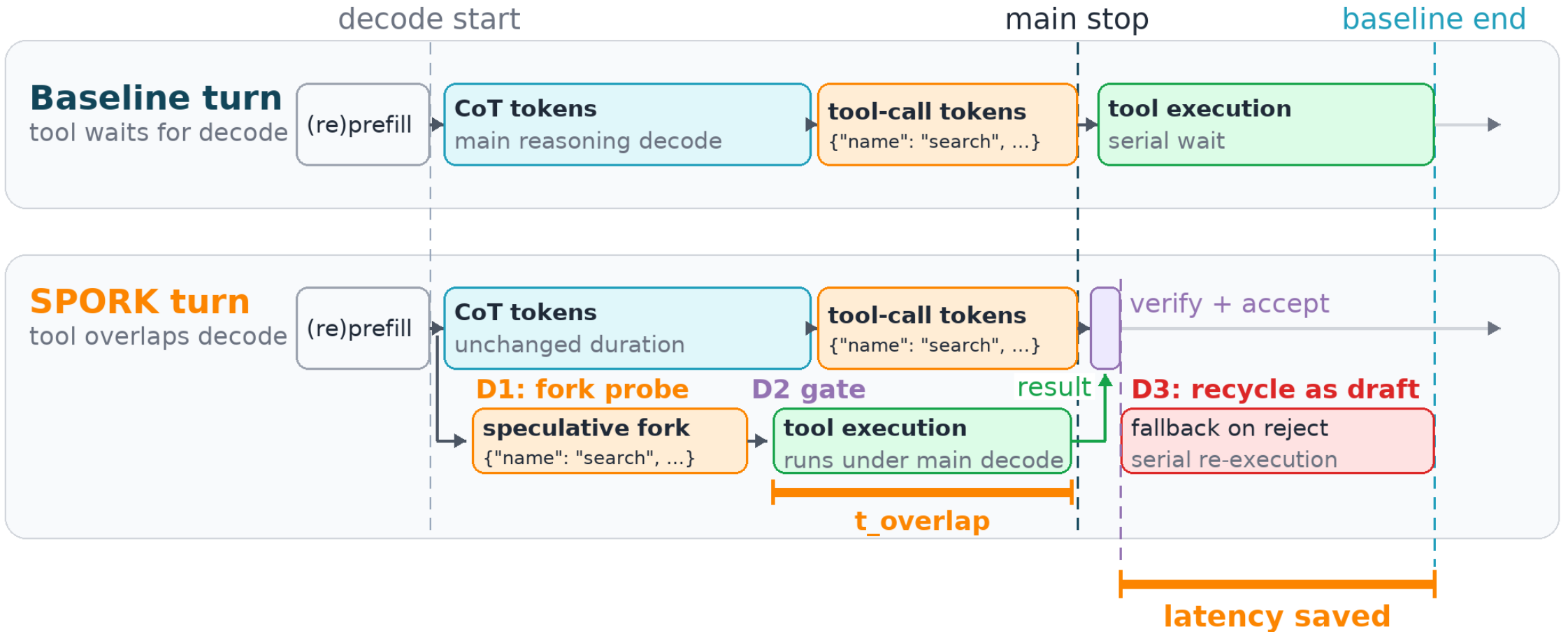
GAIA args exact
7.6% → 97.5%

Insight 3 ｜拒绝仍有前缀



中位 18 tokens
P25 = 14 tokens

SPORK 的“分叉—门控—严格验收—前缀回收”



论文原图： SPORK method overview

D1 先把“多跑一次模型”的前缀开销压到足够低

$$\text{Speedup} = \frac{T_{base}}{T^{*}_{base} - \alpha t_{overlap} + T_{oh}}$$

Break-even $\alpha\, t_{overlap} \geq T_{oh}$

α

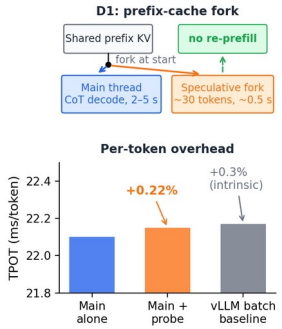
严格命中并接受已执行工具的 turn 比率。

t_overlap

工具执行与主生成真正重叠的时间。

T_oh

probe prefill/decode 以及对主流的干扰。



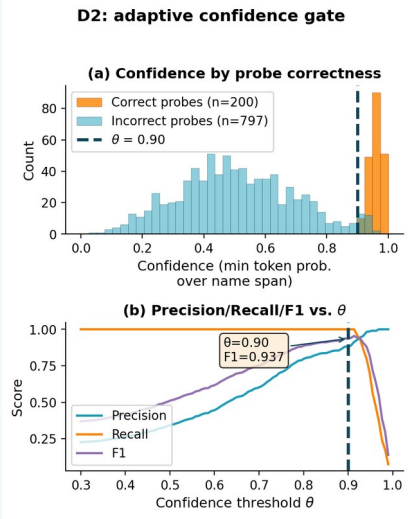
D1 的实际效果

等 main 第一个 token 后再发 probe，前缀 KV 已入缓存。
prefill 1.3s → 0.05s
整个 probe 1.6s → 0.35s
main TPOT 干扰约 +0.22%

D2 提高期望重叠， D3 降低失败分支的浪费

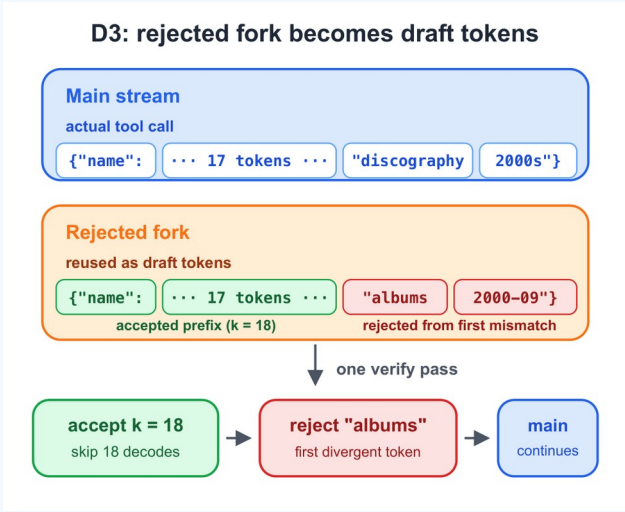
D2 | Adaptive Confidence Gate

$$c = \min_{i=2,\ldots,L} \exp(\ell_i)$$



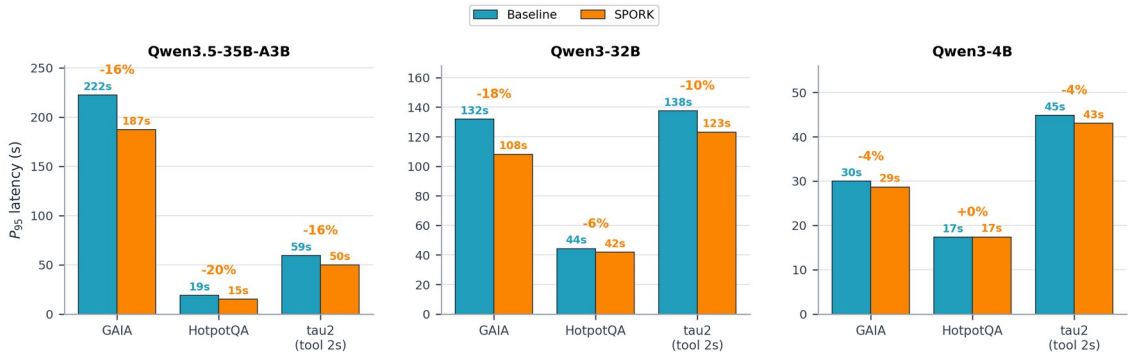
θ=0.90： Precision 88%， Recall 100%， F1=0.937；过滤 77% probes

D3 | Partial-token Accept

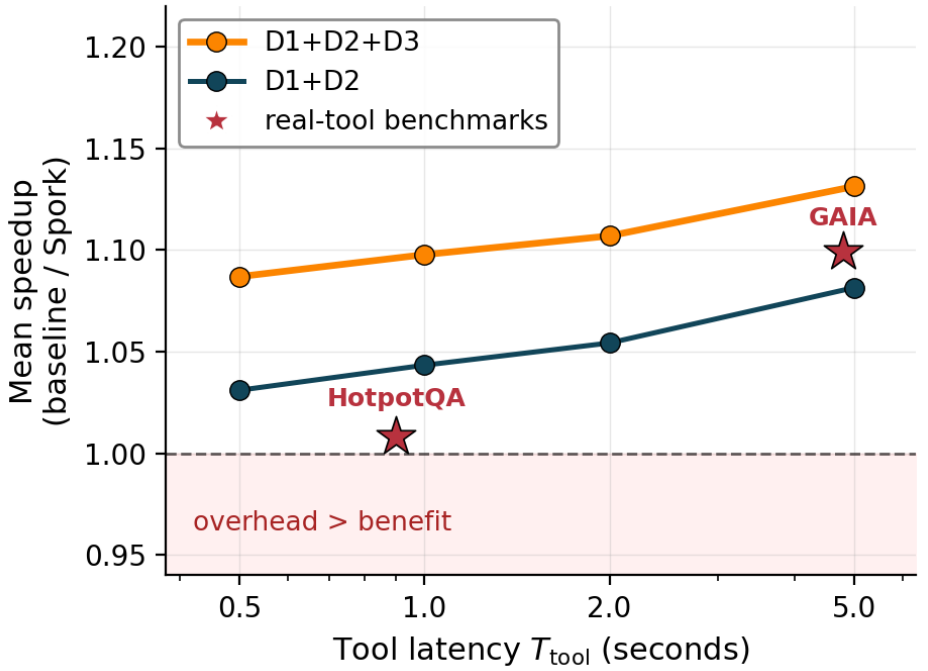


- 主流进入 tool-call opener 时，将 probe body 注入为草稿。
- 主模型并行验证，接受最长一致前缀 k 。
- 工具级失配仍获得 token 级复用。

SPORK 的收益不大但稳健：主要改善真实工具任务 P95



真实 Agent 任务 E2E/P95



何时 self-speculation 划算

-6%~18%
P95 延迟

±1 pp
任务准确率

5
主要 benchmarks

有利：工具慢 + CoT 窗口长 + 严格命中高
不利：工具快 + 参数晚确定

SPORK 是 Agent 动作投机与 Token 投机解码之间最直接的桥

三句话带走

01

预测源：当前 Actor 自身 Prefix-KV fork，无额外模型与数据。

02

两层接受：工具级完整 name+args；失败分支回到 token 级验证。

03

系统经验：用 α·t_overlap 与 T_oh 的 break-even 判断是否划算。

适用边界 / 限制

- 工具名早确定不等于完整参数早确定，严格命中仍是瓶颈。
- D3 需修改推理引擎 proposer path； HTTP 控制器只能实现 D1+D2 。
- 对副作用工具，置信门控不能替代隔离 / 确认。

Survey 技术链中的位置

PASTE
历史轨迹模式
↓
SPORK
同模型 fork + token 前缀回收